

Write a JavaScript function to simulate an ATM cash withdrawal. The ATM contains a limited number of notes for each denomination. Given a requested withdrawal amount, return the combination of notes required to dispense the amount. The ATM should prefer higher denominations first. If the exact requested amount cannot be dispensed using the available notes, return an appropriate error-- Like that give me javascript coding interview questions for experienced candidate maximum scenario based functions - with simple solution please give it here

Yes. For an experienced JavaScript interview, coding questions are often more useful when they are framed as **real application scenarios** rather than only "reverse a string" or "find a palindrome."

Below are **30 scenario-based JavaScript coding interview questions with simple solutions**. I'm keeping the code intentionally straightforward so it is easy to write and explain during an interview.

1. ATM Cash Withdrawal

Scenario

An ATM contains limited notes. Given a withdrawal amount, dispense the amount using **higher denominations first**. If the exact amount cannot be provided, return an error.

```
function withdraw(amount, atm) {
  const result = {};
  let remaining = amount;

  const denominations = Object.keys(atm)
    .map(Number)
    .sort((a, b) => b - a);

  for (let note of denominations) {
    const needed = Math.floor(remaining / note);
    const available = atm[note];
```

```

    const count = Math.min(needed, available);

    if (count > 0) {
        result[note] = count;
        remaining -= note * count;
    }
}

if (remaining !== 0) {
    return "Unable to dispense exact amount";
}

return result;
}

const atm = {
    2000: 2,
    500: 5,
    200: 10,
    100: 10
};

console.log(withdraw(3700, atm));

```

Output:

```

{
  2000: 1,
  500: 3,
  200: 1
}

```



Interview concepts: Objects, sorting, loops, greedy algorithm, limited inventory.

2. Shopping Cart Total

Scenario

An e-commerce application has a shopping cart. Calculate the total price based on product price and quantity.

```

function calculateTotal(cart) {
    let total = 0;

    for (let item of cart) {

```

```
        total += item.price * item.quantity;
    }

    return total;
}

const cart = [
    { name: "Laptop", price: 50000, quantity: 1 },
    { name: "Mouse", price: 500, quantity: 2 },
    { name: "Keyboard", price: 1000, quantity: 1 }
];

console.log(calculateTotal(cart));
```

Output:

52000



Interview concepts: Arrays, objects, iteration.

3. Apply Discount Based on Order Amount

Scenario

An e-commerce site provides:

- 10% discount for orders $\geq$ ₹5000
- 5% discount for orders $\geq$ ₹2000
- No discount otherwise

```
function calculateFinalAmount(amount) {
    let discount = 0;

    if (amount >= 5000) {
        discount = 10;
    } else if (amount >= 2000) {
        discount = 5;
    }

    return amount - (amount * discount / 100);
}

console.log(calculateFinalAmount(6000));
```

Output:



4. Find Duplicate Employee IDs

Scenario

You receive employee IDs from an API. Find duplicate IDs.

```
function findDuplicates(ids) {  
  const seen = new Set();  
  const duplicates = new Set();  
  
  for (let id of ids) {  
    if (seen.has(id)) {  
      duplicates.add(id);  
    } else {  
      seen.add(id);  
    }  
  }  
  
  return [...duplicates];  
}  
  
console.log(findDuplicates([101, 102, 103, 102, 104, 101]));
```

Output:

```
[102, 101]
```



Interview concepts: Set , duplicate detection, O(n).

5. Remove Duplicate Records

Scenario

An API returns duplicate users. Remove duplicates based on `id` .

```
function removeDuplicates(users) {  
  const map = new Map();
```

```
    for (let user of users) {  
        map.set(user.id, user);  
    }  
  
    return [...map.values()];  
}  
  
const users = [  
    { id: 1, name: "John" },  
    { id: 2, name: "David" },  
    { id: 1, name: "John" }  
];  
  
console.log(removeDuplicates(users));
```

6. Group Employees by Department

Scenario

Group employees based on department.

```
function groupByDepartment(employees) {  
    return employees.reduce((result, employee) => {  
  
        if (!result[employee.department]) {  
            result[employee.department] = [];  
        }  
  
        result[employee.department].push(employee);  
  
        return result;  
  
    }, {});  
}  
  
const employees = [  
    { name: "John", department: "IT" },  
    { name: "David", department: "HR" },  
    { name: "Peter", department: "IT" }  
];  
  
console.log(groupByDepartment(employees));
```

Interview concepts: `reduce()` , grouping, object manipulation.

7. Find Highest Paid Employee

Scenario

Find the employee with the highest salary.

```
function highestPaidEmployee(employees) {  
  return employees.reduce((highest, employee) => {  
    return employee.salary > highest.salary  
      ? employee  
      : highest;  
  });  
}  
  
const employees = [  
  { name: "John", salary: 50000 },  
  { name: "David", salary: 80000 },  
  { name: "Peter", salary: 60000 }  
];  
  
console.log(highestPaidEmployee(employees));
```

Output:

```
{ name: "David", salary: 80000 }
```



8. Find Second Highest Salary

Scenario

HR wants the second-highest **unique** salary.

```
function secondHighestSalary(employees) {  
  const salaries = employees.map(x => x.salary);  
  
  const unique = [...new Set(salaries)];  
  
  unique.sort((a, b) => b - a);  
  
  return unique[1];  
}  
  
const employees = [
```

```
{ name: "A", salary: 50000 },
{ name: "B", salary: 80000 },
{ name: "C", salary: 60000 },
{ name: "D", salary: 80000 }
];

console.log(secondHighestSalary(employees));
```

Output:

60000



9. Count Product Occurrences

Scenario

A list contains purchased products. Find how many times each product was purchased.

```
function countProducts(products) {
  const result = {};

  for (let product of products) {
    result[product] = (result[product] || 0) + 1;
  }

  return result;
}

console.log(
  countProducts(["Laptop", "Mouse", "Laptop", "Keyboard", "Mouse"])
);
```

Output:

```
{
  Laptop: 2,
  Mouse: 2,
  Keyboard: 1
}
```



10. Inventory Stock Validation

Scenario

Before placing an order, verify whether sufficient stock is available.

```
function checkStock(product, requestedQuantity) {  
  
    if (requestedQuantity <= product.stock) {  
        return "Stock available";  
    }  
  
    return "Insufficient stock";  
}  
  
const product = {  
    name: "Laptop",  
    stock: 5  
};  
  
console.log(checkStock(product, 3));
```

11. Bank Account Transaction

Scenario

Implement basic deposit and withdrawal operations.

```
function bankTransaction(balance, type, amount) {  
  
    if (amount <= 0) {  
        return "Invalid amount";  
    }  
  
    if (type === "deposit") {  
        return balance + amount;  
    }  
  
    if (type === "withdraw") {  
  
        if (amount > balance) {  
            return "Insufficient balance";  
        }  
  
        return balance - amount;  
    }  
}
```

```
    return "Invalid transaction";
  }

  console.log(bankTransaction(10000, "withdraw", 3000));
```

Output:

7000



12. Transfer Money Between Accounts

Scenario

Transfer money from one account to another.

```
function transfer(fromAccount, toAccount, amount) {

  if (amount <= 0) {
    return "Invalid amount";
  }

  if (fromAccount.balance < amount) {
    return "Insufficient balance";
  }

  fromAccount.balance -= amount;
  toAccount.balance += amount;

  return {
    fromAccount,
    toAccount
  };
}

const account1 = {
  name: "John",
  balance: 10000
};

const account2 = {
  name: "David",
  balance: 5000
};
```

```
console.log(transfer(account1, account2, 3000));
```

Interview follow-up: What happens if updating the second account fails? This leads naturally into transaction/rollback concepts.

13. Login Attempt Lock

Scenario

Lock a user after three failed login attempts.

```
function login(user, password) {

  if (user.isLocked) {
    return "Account locked";
  }

  if (user.password === password) {
    user.failedAttempts = 0;
    return "Login successful";
  }

  user.failedAttempts++;

  if (user.failedAttempts >= 3) {
    user.isLocked = true;
    return "Account locked";
  }

  return "Invalid password";
}

const user = {
  password: "1234",
  failedAttempts: 0,
  isLocked: false
};

console.log(login(user, "1111"));
console.log(login(user, "2222"));
console.log(login(user, "3333"));
```

14. Password Strength Validation

Scenario

Validate that a password contains uppercase, lowercase, number, and minimum 8 characters.

```
function validatePassword(password) {  
  
    if (password.length < 8) {  
        return false;  
    }  
  
    const hasUpper = /[A-Z]/.test(password);  
    const hasLower = /[a-z]/.test(password);  
    const hasNumber = /[0-9]/.test(password);  
  
    return hasUpper && hasLower && hasNumber;  
}  
  
console.log(validatePassword("Hello123"));
```

Output:

true



15. Mask Credit Card Number

Scenario

Display only the last four digits of a card.

```
function maskCard(cardNumber) {  
    const lastFour = cardNumber.slice(-4);  
  
    return "*".repeat(cardNumber.length - 4) + lastFour;  
}  
  
console.log(maskCard("1234567812345678"));
```

Output:

*****5678



16. Search Products

Scenario

Implement a simple case-insensitive product search.

```
function searchProducts(products, searchText) {  
  
    return products.filter(product =>  
        product.name  
            .toLowerCase()  
            .includes(searchText.toLowerCase())  
    );  
}  
  
const products = [  
    { id: 1, name: "Apple iPhone" },  
    { id: 2, name: "Samsung Phone" },  
    { id: 3, name: "Apple MacBook" }  
];  
  
console.log(searchProducts(products, "apple"));
```

17. Sort Products by Price

Scenario

Allow users to sort products by price.

```
function sortByPrice(products, direction) {  
  
    return [...products].sort((a, b) => {  
  
        if (direction === "asc") {  
            return a.price - b.price;  
        }  
  
        return b.price - a.price;  
    });  
}  
  
const products = [
```

```
{ name: "A", price: 500 },
{ name: "B", price: 200 },
{ name: "C", price: 800 }
];

console.log(sortByPrice(products, "asc"));
```

Important: [...products] avoids modifying the original array.

18. Pagination Function

Scenario

An API returns 10,000 records. Display 10 records per page.

```
function paginate(data, pageNumber, pageSize) {

  const start = (pageNumber - 1) * pageSize;
  const end = start + pageSize;

  return data.slice(start, end);
}

const data = [
  1, 2, 3, 4, 5,
  6, 7, 8, 9, 10,
  11, 12, 13, 14, 15
];

console.log(paginate(data, 2, 5));
```

Output:

```
[6, 7, 8, 9, 10]
```



19. Flatten Nested Categories

Scenario

An API returns nested category arrays.

```
function flattenArray(arr) {
  return arr.flat(Infinity);
}

const categories = [
  ["Electronics", "Mobile"],
  ["Clothes", ["Men", "Women"]]
];

console.log(flattenArray(categories));
```

Output:

```
["Electronics", "Mobile", "Clothes", "Men", "Women"]
```



Without flat()

```
function flattenArray(arr) {
  let result = [];

  for (let item of arr) {
    if (Array.isArray(item)) {
      result = result.concat(flattenArray(item));
    } else {
      result.push(item);
    }
  }

  return result;
}
```

The recursive version is more interesting in an experienced interview.

20. Retry Failed API

Scenario

An external API occasionally fails. Retry it a maximum of three times.

```
async function retryApi(apiFunction, maxAttempts = 3) {

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
```

```

    try {
      return await apiFunction();
    }
    catch (error) {

      if (attempt === maxAttempts) {
        throw error;
      }

      console.log("Retrying...");
    }
  }
}

```

Example:

```

async function getData() {

  const response = await fetch("/api/products");

  if (!response.ok) {
    throw new Error("API failed");
  }

  return response.json();
}

retryApi(getData, 3);

```

Experienced follow-up: Should every failed request be retried? No. Be careful with non-idempotent operations such as payment creation.

21. API Timeout

Scenario

An external API sometimes takes too long. Fail if it doesn't respond within five seconds.

```

function timeout(ms) {
  return new Promise( (_, reject) => {
    setTimeout(() => {
      reject(new Error("Request timeout"));
    }, ms);
  });
}

```

```
    });  
  }  
  
  async function callApi() {  
  
    return Promise.race([  
      fetch("/api/products"),  
      timeout(5000)  
    ]);  
  }  
}
```

Interview concepts: Promise, Promise.race() , timeout handling.

22. Run Multiple APIs in Parallel

Scenario

A dashboard requires user, order, and notification information. The APIs are independent.

```
async function loadDashboard() {  
  
  const [user, orders, notifications] =  
    await Promise.all([  
      fetch("/api/user"),  
      fetch("/api/orders"),  
      fetch("/api/notifications")  
    ]);  
  
  return {  
    user,  
    orders,  
    notifications  
  };  
}
```

Interview question

Why use Promise.all() ?

Because independent operations can execute concurrently instead of waiting for each one sequentially.

23. Handle Partial API Failure

Scenario

A dashboard calls three APIs. Even if one fails, display results from the others.

```
async function loadDashboard() {  
  
    const results = await Promise.allSettled([  
        fetch("/api/user"),  
        fetch("/api/orders"),  
        fetch("/api/notifications")  
    ]);  
  
    return results;  
}
```

Interview follow-up:

`Promise.all()` → rejects when one promise rejects.

`Promise.allSettled()` → waits for all operations and reports individual success/failure.

24. Debounce Search

Scenario

A search API should not be called for every keyboard press. Call it only after the user stops typing.

```
function debounce(func, delay) {  
  
    let timer;  
  
    return function (...args) {  
  
        clearTimeout(timer);  
  
        timer = setTimeout(() => {  
            func(...args);  
        }, delay);  
    };  
}  
  
function search(text) {  
    console.log("Calling API:", text);  
}
```

```
}

const searchWithDebounce = debounce(search, 500);

searchWithDebounce("a");
searchWithDebounce("ap");
searchWithDebounce("app");
```

Only the final call executes after 500ms.

Very important experienced JavaScript question.

25. Throttle Button/API Calls

Scenario

A button should execute at most once every two seconds.

```
function throttle(func, delay) {

  let allowed = true;

  return function (...args) {

    if (!allowed) {
      return;
    }

    func(...args);

    allowed = false;

    setTimeout(() => {
      allowed = true;
    }, delay);
  };
}

function save() {
  console.log("Saving...");
}

const throttledSave = throttle(save, 2000);
```

Difference

Debounce: Execute after activity stops.

Throttle: Execute at most once during a specified interval.

26. Prevent Duplicate API Submission

Scenario

A user double-clicks the **Place Order** button. Prevent duplicate API requests.

```
let isSubmitting = false;

async function placeOrder(order) {

  if (isSubmitting) {
    return;
  }

  isSubmitting = true;

  try {

    const response = await fetch("/api/orders", {
      method: "POST",
      body: JSON.stringify(order)
    });

    return response;

  } finally {

    isSubmitting = false;
  }
}
```

Experienced discussion: Frontend protection alone isn't sufficient. The backend should also support idempotency/duplicate-request protection.

27. Simple In-Memory Cache

Scenario

Avoid repeatedly calling an API for the same data.

```
const cache = {};  
  
async function getUser(id) {  
  
    if (cache[id]) {  
        console.log("Returning from cache");  
        return cache[id];  
    }  
  
    const response = await fetch(`/api/users/${id}`);  
    const user = await response.json();  
  
    cache[id] = user;  
  
    return user;  
}
```

Interview follow-up

What's missing?

Cache expiration.

28. Cache With Expiration

Scenario

Cache API results for one minute.

```
const cache = {};  
  
async function getUser(id) {  
  
    const cached = cache[id];  
  
    if (cached && Date.now() < cached.expiry) {  
        return cached.data;  
    }  
  
    const response = await fetch(`/api/users/${id}`);  
    const data = await response.json();  
  
    cache[id] = {  
        data: data,  
        expiry: Date.now() + 60000,  
    };  
}
```

```
    expiry: Date.now() + 60000
  };

  return data;
}
```

Interview concepts: Caching, TTL, performance optimization.

29. Find Missing Number

Scenario

Invoice numbers should be sequential from 1 to N, but one is missing.

```
function findMissingNumber(numbers, n) {

  const expected = n * (n + 1) / 2;

  const actual = numbers.reduce(
    (sum, number) => sum + number,
    0
  );

  return expected - actual;
}

console.log(findMissingNumber([1, 2, 3, 5, 6], 6));
```

Output:

4



30. Find First Non-Repeating Character

Scenario

Find the first character that occurs only once.

```
function firstUniqueCharacter(text) {

  const count = {};
```

```

    for (let char of text) {
      count[char] = (count[char] || 0) + 1;
    }

    for (let char of text) {
      if (count[char] === 1) {
        return char;
      }
    }

    return null;
  }

  console.log(firstUniqueCharacter("aabbcdde"));

```

Output:

c



31. Merge Orders With Customer Information

Scenario

Two APIs return separate customer and order arrays. Combine them.

```

function mergeData(customers, orders) {

  return orders.map(order => {

    const customer = customers.find(
      c => c.id === order.customerId
    );

    return {
      ...order,
      customerName: customer?.name
    };
  });
}

const customers = [
  { id: 1, name: "John" },
  { id: 2, name: "David" }
];

```

```
const orders = [
  { id: 101, customerId: 1, amount: 500 },
  { id: 102, customerId: 2, amount: 1000 }
];

console.log(mergeData(customers, orders));
```

32. Calculate Department Salary Total

Scenario

Calculate total salary for each department.

```
function departmentSalary(employees) {

  return employees.reduce((result, employee) => {

    result[employee.department] =
      (result[employee.department] || 0)
      + employee.salary;

    return result;

  }, {});

}

const employees = [
  { name: "A", department: "IT", salary: 50000 },
  { name: "B", department: "HR", salary: 40000 },
  { name: "C", department: "IT", salary: 60000 }
];

console.log(departmentSalary(employees));
```

Output:

```
{
  IT: 110000,
  HR: 40000
}
```



33. Shopping Cart Quantity Update

Scenario

If a product already exists in the cart, increase its quantity. Otherwise add it.

```
function addToCart(cart, product) {

  const existing = cart.find(
    item => item.id === product.id
  );

  if (existing) {
    existing.quantity++;
  } else {
    cart.push({
      ...product,
      quantity: 1
    });
  }

  return cart;
}

const cart = [
  { id: 1, name: "Laptop", quantity: 1 }
];

console.log(
  addToCart(cart, { id: 1, name: "Laptop" })
);
```

34. Role-Based Access

Scenario

Check whether a user has permission to perform an operation.

```
function hasPermission(user, requiredPermission) {

  return user.permissions.includes(requiredPermission);
}

const user = {
  name: "John",
  permissions: [
```

```
        "VIEW_USER",
        "CREATE_USER",
        "EDIT_USER"
    ]
};

console.log(
    hasPermission(user, "DELETE_USER")
);
```

Output:

false



35. Convert API Array to Lookup Object

Scenario

You repeatedly search thousands of products by ID. Convert the array to a lookup object for faster access.

```
function createProductLookup(products) {

    return products.reduce((result, product) => {

        result[product.id] = product;

        return result;

    }, {});
}

const products = [
    { id: 101, name: "Laptop" },
    { id: 102, name: "Mouse" },
    { id: 103, name: "Keyboard" }
];

const lookup = createProductLookup(products);

console.log(lookup[102]);
```

Output:

```
{ id: 102, name: "Mouse" }
```



This is a good experienced-level discussion because repeated `.find()` calls are $O(n)$, whereas lookup by key is approximately $O(1)$.

36. Order Status Validation

Scenario

An order can follow:

```
Pending → Processing → Shipped → Delivered
```



Don't allow invalid transitions.

```
function changeOrderStatus(currentStatus, newStatus) {

  const transitions = {
    Pending: ["Processing"],
    Processing: ["Shipped"],
    Shipped: ["Delivered"],
    Delivered: []
  };

  if (transitions[currentStatus]?.includes(newStatus)) {
    return newStatus;
  }

  return "Invalid status transition";
}

console.log(
  changeOrderStatus("Pending", "Processing")
);

console.log(
  changeOrderStatus("Pending", "Delivered")
);
```

This is an excellent scenario question for experienced candidates because it resembles real workflow/state-machine logic.

37. Validate Booking Date Overlap

Scenario

A meeting room already has bookings. Check whether a new booking overlaps an existing booking.

```
function hasOverlap(bookings, newBooking) {  
  
    return bookings.some(booking =>  
        newBooking.start < booking.end &&  
        newBooking.end > booking.start  
    );  
}  
  
const bookings = [  
    { start: 10, end: 11 },  
    { start: 13, end: 14 }  
];  
  
console.log(  
    hasOverlap(bookings, { start: 10.5, end: 11.5 })  
);
```

Output:

true



38. Process Queue One by One

Scenario

Files need to be uploaded sequentially instead of all at once.

```
async function processQueue(files) {  
  
    for (let file of files) {  
  
        await uploadFile(file);  
  
        console.log("Uploaded:", file);  
    }  
}
```

```
async function uploadFile(file) {
  return new Promise(resolve => {
    setTimeout(resolve, 1000);
  });
}

processQueue(["file1.pdf", "file2.pdf", "file3.pdf"]);
```

Interview follow-up: Compare this with `Promise.all()` .

39. Memoization

Scenario

A calculation is expensive and repeatedly called with the same input.

```
function memoize(fn) {

  const cache = {};

  return function (value) {

    if (value in cache) {
      return cache[value];
    }

    const result = fn(value);

    cache[value] = result;

    return result;
  };
}

function square(number) {
  console.log("Calculating...");
  return number * number;
}

const cachedSquare = memoize(square);

console.log(cachedSquare(10));
console.log(cachedSquare(10));
```

The calculation happens only once.

40. Retry API With Delay

Scenario

An external service fails temporarily. Retry three times with a delay between attempts.

```
function delay(ms) {
  return new Promise(resolve =>
    setTimeout(resolve, ms)
  );
}

async function retry(fn, attempts = 3) {
  for (let i = 1; i <= attempts; i++) {
    try {
      return await fn();
    }
    catch (error) {
      if (i === attempts) {
        throw error;
      }

      await delay(1000);
    }
  }
}
```

This is particularly useful for senior/full-stack interviews.

41. Find Top 3 Selling Products

Scenario

Given sales records, return the top three products based on quantity sold.

```
function topSellingProducts(sales) {

  const totals = {};

  for (let sale of sales) {
```

```

        totals[sale.product] =
            (totals[sale.product] || 0)
            + sale.quantity;
    }

    return Object.entries(totals)
        .sort((a, b) => b[1] - a[1])
        .slice(0, 3);
}

const sales = [
    { product: "Laptop", quantity: 5 },
    { product: "Mouse", quantity: 20 },
    { product: "Laptop", quantity: 10 },
    { product: "Keyboard", quantity: 12 },
    { product: "Monitor", quantity: 8 }
];

console.log(topSellingProducts(sales));

```

42. Detect Duplicate API Requests

Scenario

Prevent the same request from being processed repeatedly.

```

const processingRequests = new Set();

async function processRequest(requestId) {

    if (processingRequests.has(requestId)) {
        return "Duplicate request";
    }

    processingRequests.add(requestId);

    try {

        console.log("Processing:", requestId);

        // API operation

        return "Success";

    } finally {

        processingRequests.delete(requestId);
    }
}

```

```
}  
}
```

This can lead to a senior-level discussion about **idempotency keys** and server-side duplicate prevention.

43. Calculate Age From DOB

Scenario

Calculate a customer's age.

```
function calculateAge(dateOfBirth) {  
  
    const dob = new Date(dateOfBirth);  
    const today = new Date();  
  
    let age =  
        today.getFullYear() - dob.getFullYear();  
  
    const monthDifference =  
        today.getMonth() - dob.getMonth();  
  
    if (  
        monthDifference < 0 ||  
        (  
            monthDifference === 0 &&  
            today.getDate() < dob.getDate()  
        )  
    ) {  
        age--;  
    }  
  
    return age;  
}  
  
console.log(calculateAge("1990-10-15"));
```

44. Find Difference Between Two Arrays

Scenario

Find users removed between the previous and current API response.

```
function findRemovedUsers(oldUsers, newUsers) {

  const newIds = new Set(
    newUsers.map(user => user.id)
  );

  return oldUsers.filter(
    user => !newIds.has(user.id)
  );
}

const oldUsers = [
  { id: 1 },
  { id: 2 },
  { id: 3 }
];

const newUsers = [
  { id: 1 },
  { id: 3 }
];

console.log(
  findRemovedUsers(oldUsers, newUsers)
);
```

Output:

```
[{ id: 2 }]
```



45. Chunk Large Array

Scenario

You have 10,000 records, but an API accepts only 100 records per request.

```
function chunkArray(data, size) {

  const result = [];

  for (let i = 0; i < data.length; i += size) {

    result.push(
```

```
        data.slice(i, i + size)
    );
}

return result;
}

console.log(
    chunkArray([1, 2, 3, 4, 5, 6, 7], 3)
);
```

Output:

```
[
  [1, 2, 3],
  [4, 5, 6],
  [7]
]
```



This is a very practical batch-processing interview scenario.

Most Important Ones for Experienced Interviews

If you're preparing for a **Senior .NET Full Stack / React / JavaScript** interview, I would practice these especially:

Priority	Scenario	Main JavaScript Concept
★ ★ ★	ATM withdrawal	Algorithm + objects
★ ★ ★	Debounce search	Closures + timers
★ ★ ★	Throttle	Closures + timers
★ ★ ★	Retry failed API	async/await
★ ★ ★	API timeout	Promise.race
★ ★ ★	Parallel APIs	Promise.all
★ ★ ★	Partial API failures	Promise.allSettled
★ ★ ★	Prevent duplicate submit	Async state
★ ★ ★	Cache with expiry	Objects + Date

Priority	Scenario	Main JavaScript Concept
★ ★ ★	Pagination	Array slicing
★ ★ ★	Group employees	reduce
★ ★ ★	Remove duplicates	Map / Set
★ ★ ★	Second highest salary	Array + Set
★ ★ ★	Booking overlap	Array.some
★ ★ ★	Order status workflow	Business logic
★ ★ ★	Batch processing	Async + arrays
★ ★ ★	Memoization	Closures + caching
★ ★ ★	Array lookup optimization	Map/Object
★ ★ ★	Money transfer	Business logic
★ ★ ★	Role permissions	Array methods

For an experienced candidate, don't prepare only the code. Be ready for the interviewer to change the requirement after you finish—for example: **“What if there are 100,000 records?”**, **“What if the API fails?”**, **“How do you avoid modifying the original array?”**, **“What is the time complexity?”**, **“Can you do it without `sort()` ?”**, **“What happens if two users submit simultaneously?”** Those follow-up questions are often where senior-level evaluation happens.








[Convert to PDF](#)

ChatGPT can make mistakes. Check important info.